

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-приложения для фитнес-клуба с системой абонементов, отслеживанием тренировок и личными кабинетами пользователей»

Выполнил: Демидов Матвей Вячеславович

Группа: 241-326

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

**«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)**

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /

«____» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Демидову Матвею Вячеславовичу,

обучающемуся группы 241-326,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-приложения для фитнес-клуба с системой абонементов, отслеживанием тренировок и личными кабинетами пользователей»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих программных продуктов по теме работы	10	22.03.2026	
Задача 1.2. Анализ программных инструментов разработки веб-приложений	7	26.03.2026	
Задача 1.3. Формулировка цели и задач работы	8	30.03.2026	
Раздел 2. Проектирование веб-приложения			
Задача 2.1. Анализ целевой аудитории	5	02.04.2026	
Задача 2.2. Описание функциональности приложения (диаграмма вариантов использования, user story)	7	06.04.2026	

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.3. Проектирование модели данных (ER-диаграмма, логическая и физическая схемы БД)	8	10.04.2026	
Задача 2.4. Разработка макетов страниц (Wireframe)	5	14.04.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Разработка базовой структуры приложения и вёрстка шаблонов страниц	8	21.04.2026	
Задача 3.2. Реализация аутентификации и авторизации с разграничением ролей	7	27.04.2026	
Задача 3.3. Реализация CRUD-интерфейса для управления пользователями, абонентами и тренировками	7	03.05.2026	
Задача 3.4. Реализация системы абонентов и отслеживания тренировок	12	09.05.2026	
Задача 3.5. Реализация загрузки файлов (фото прогресса, аватары) и выгрузки отчётов	6	15.05.2026	
Раздел 4. Оформление итогов работы			
Задача 4.1. Создание Git-репозитория с кодом проекта	3	22.03.2026	
Задача 4.2. Деплой приложения на хостинг	4	26.05.2026	
Задача 4.3. Оформление отчёта о проделанной работе	3	26.05.2026	

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«30» марта 2026 г.

(подпись) А. С. Кружалов

Дата выдачи задания

«30» марта 2026 г.

Дата сдачи выполненной работы

«13» июня 2026 г.

Задание принял к исполнению

«31» марта 2026 г.

(подпись) М. В. Демидов

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	6
1.1 Обзор существующих программных продуктов по теме работы	6
1.2 Анализ программных инструментов разработки веб-приложений	7
1.3 Формулировка цели и задач работы	7
ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ	9
2.1 Анализ целевой аудитории и формирование требований	9
2.2 Описание функциональности платформы	9
2.3 Проектирование модели данных	11
2.4 Разработка макетов страниц	12
ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ	16
3.1 Разработка базовой структуры клиентской части приложения .	16
3.2 Разработка серверной части, реализация аутентификации и авторизации	17
3.3 Реализация CRUD-интерфейсов для пользователей, абонен- тов и тренировок	18
3.4 Реализация системы записи на занятия и бизнес-логики списа- ния визитов	19
3.5 Разработка модуля отчетности и работы с файлами	19
ЗАКЛЮЧЕНИЕ	21
СПИСОК ИСТОЧНИКОВ ЛИТЕРАТУРЫ	23

ВВЕДЕНИЕ

Актуальность темы. В условиях активного перехода населения к здоровому образу жизни и цифровизации сферы услуг, эффективность работы фитнес-клубов напрямую зависит от качества автоматизации взаимодействия с клиентами. Традиционные методы учета (бумажные журналы, электронные таблицы, физические пластиковые карты) приводят к снижению скорости обслуживания, ошибкам в учете абонементов и отсутствию персонализированного подхода к клиенту. Разработка специализированных веб-приложений, интегрирующих инструменты покупки абонементов, записи на занятия и отслеживания спортивного прогресса в едином личном кабинете, является необходимым условием для повышения лояльности клиентов и конкурентоспособности фитнес-клуба.

Степень разработанности проблемы. На рынке существует множество систем управления взаимоотношениями с клиентами (CRM) для фитнес-индустрии (например, 1С:Фитнес клуб, Mobifitness, YCLIENTS). Однако большинство из них представляют собой тяжеловесные корпоративные решения, требующие сложного внедрения, высоких финансовых затрат и избыточные для малых и средних студий. Кроме того, многие из них слабо ориентированы на конечного пользователя (клиента) в части отслеживания личного тренировочного прогресса и биометрии. Создание гибких, легковесных и технологичных SaaS-решений остается актуальной практической задачей.

Объект исследования — процессы организации обслуживания клиентов, администрирования расписания и учета абонементов в фитнес-клубах.

Предмет исследования — архитектурные подходы, программные средства и алгоритмические решения для автоматизации взаимодействия фитнес-клуба и клиентов в рамках единой веб-ориентированной платформы.

Целью данной работы является проектирование и реализация клиент-серверного веб-приложения (SaaS-платформы), предназначенного для автоматизации продажи и учета абонементов, управления расписанием, а также предоставления пользователям личных кабинетов для мониторинга тренировочного прогресса и выгрузки аналитики.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих программных продуктов по теме работы

На современном рынке программного обеспечения для фитнес-индустрии выделяются универсальные CRM-системы и специализированные платформы. Для выявления их сильных и слабых сторон проведем структурированный анализ на основе набора критериев, отражающих потребности малых и средних фитнес-клубов, а также самих клиентов.

Критерии сравнения:

1. **Учет абонементов и посещений:** наличие гибкой системы тарифов.
2. **Онлайн-запись:** возможность самостоятельной записи клиента.
3. **Трекинг прогресса:** личный кабинет клиента с историей тренировок и замерами.
4. **Легковесность и доступность:** низкий порог входа для малого бизнеса, отсутствие перегруженного функционала (сложной бухгалтерии).
5. **Наличие ролевой модели:** разделение интерфейсов для администратора, тренера и клиента.

Сравнение ключевых игроков представлено в таблице 1.1.

Таблица 1.1 – Сравнительный анализ платформ для фитнес-клубов

Название	Учет абонементов	Онлайн-запись	Трекинг прогресса	Легковесность	Ролевая модель
1С:Фитнес клуб	Есть	Есть	Нет	Нет	Есть
Mobifitness	Есть	Есть	Базовый	Нет	Есть
YCLIENTS	Базовый	Есть	Нет	Да	Частично
Наше ПО	Есть	Есть	Детальный	Да	Есть

Вывод: Анализ показал, что популярные продукты (1С, Mobifitness) избыточны для небольших студий и имеют высокую стоимость внедрения. Легкие системы (YCLIENTS) закрывают потребность в записи, но игнорируют спортивную составляющую (отслеживание прогресса клиента). Разрабатываемое приложение объединит простоту учета абонементов с персонализированным трекингом тренировок, закрывая пустующую нишу.

1.2 Анализ программных инструментов разработки веб-приложений

Для реализации проекта рассматриваются современные полнофункциональные (Fullstack) подходы создания клиент-серверных приложений. Выбор стека технологий обусловлен требованиями к интерактивности пользовательского интерфейса, безопасности транзакций и скорости разработки:

- **Frontend-фреймворк и библиотеки:** Для создания клиентской части приложения выбрана библиотека **React.js**. Она обеспечивает высокую скорость рендеринга благодаря Virtual DOM. Для выполнения сетевых запросов и прозрачной работы с JWT-токенами выбрана библиотека **Axios**, поддерживающая удобный механизм перехватчиков (interceptors). Стилизация интерфейсов и сложных аналитических дашбордов будет осуществляться с помощью фреймворка **Tailwind CSS**, реализующего концепцию Utility-First, что позволяет минимизировать объем CSS-кода.
- **Backend-платформа:** В качестве серверной среды исполнения выбран **Node.js** с использованием микрофреймворка **Express**. Использование единого языка (JavaScript) на клиенте и сервере существенно ускоряет разработку. Для взаимодействия с базой данных используется ORM-библиотека **Sequelize**. Она позволяет абстрагироваться от сырых SQL-запросов, использовать объектно-ориентированные модели таблиц и надежно защищает систему от атак типа SQL-инъекций.
- **Система управления базами данных (СУБД):** Для хранения информации о пользователях, абонементов и расписании тренировок выбрана реляционная СУБД **PostgreSQL**. Логика автоматического списания занятий требует строгой консистентности данных и поддержки ACID-транзакций, с чем PostgreSQL справляется эффективнее NoSQL-решений. Также она обладает нативной поддержкой типа JSONB для хранения динамических характеристик спортивного прогресса.

1.3 Формулировка цели и задач работы

На основе проведенного анализа предметной области и выбранного стека технологий была сформулирована цель исследования.

Цель работы — создание SaaS-экосистемы для фитнес-клуба, обеспечивающей удобный учет абонементов, интеллектуальную запись на занятия и ведение личных кабинетов с историей тренировочного прогресса.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Провести анализ предметной области, выявить недостатки существующих решений и обосновать выбор программных инструментов;
2. Спроектировать архитектуру базы данных (логическую и физическую модели) для хранения информации о пользователях, абонементов и расписании;
3. Разработать макеты страниц (Wireframes) и спроектировать пользовательские интерфейсы для различных ролей;
4. Реализовать клиент-серверное приложение, включая модули авторизации, управления абонементом, CRUD-интерфейсы расписания и модуль загрузки файлов;
5. Реализовать бизнес-логику транзакционного списания занятий и выгрузки аналитической отчетности;
6. Подготовить проект к развертыванию: создать Git-репозиторий и осуществить деплой веб-приложения на хостинг.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Анализ целевой аудитории и формирование требований

Анализ целевой аудитории позволяет выделить два основных сегмента пользователей фитнес-клуба. Первый сегмент — это клиенты, заинтересованные в приобретении абонементов и онлайн-записи на тренировки. Их главная потребность заключается в удобном поиске занятий по специфическим характеристикам (направление, тренер, время) и возможности отслеживания своего прогресса. Второй сегмент — тренеры и администраторы, нуждающиеся в доступном инструменте для ведения базы клиентов и контроля расписания.

Исходя из потребностей целевой аудитории, в проектируемой системе выделяются три ключевые ролевые модели с изолированными правами доступа:

- **Клиент** — авторизованный пользователь, имеющий доступ к витрине направлений. Нуждается в инструментах фильтрации, управления своими абонементом и отслеживания записей через личный кабинет.
- **Тренер** — сотрудник, управляющий своим расписанием. Основные потребности: просмотр списка записавшихся на тренировку клиентов, отметка посещаемости и ведение журнала прогресса подопечных.
- **Администратор** — сотрудник платформы, обладающий полными правами. Отвечает за модерацию контента, продажу абонементов, управление сеткой расписания и выгрузку агрегированной статистики.

На основе выделенных ролей был сформирован перечень различных требований к разрабатываемому программному продукту. Основные из них: реализация ролевой модели авторизации, создание CRUD-интерфейсов для управления расписанием и агрегация данных для формирования отчетности.

2.2 Описание функциональности платформы

Для формализации функциональных требований и проектирования бизнес-логики разрабатываемой веб-платформы применена методология пользовательских историй. Данная модель визуализирует границы системы и детализирует архитектуру взаимодействия ролей с использованием механизмов наследования и расширенной нотации связей.

На рисунке 2.1 представлена диаграмма вариантов использования (Use Case).

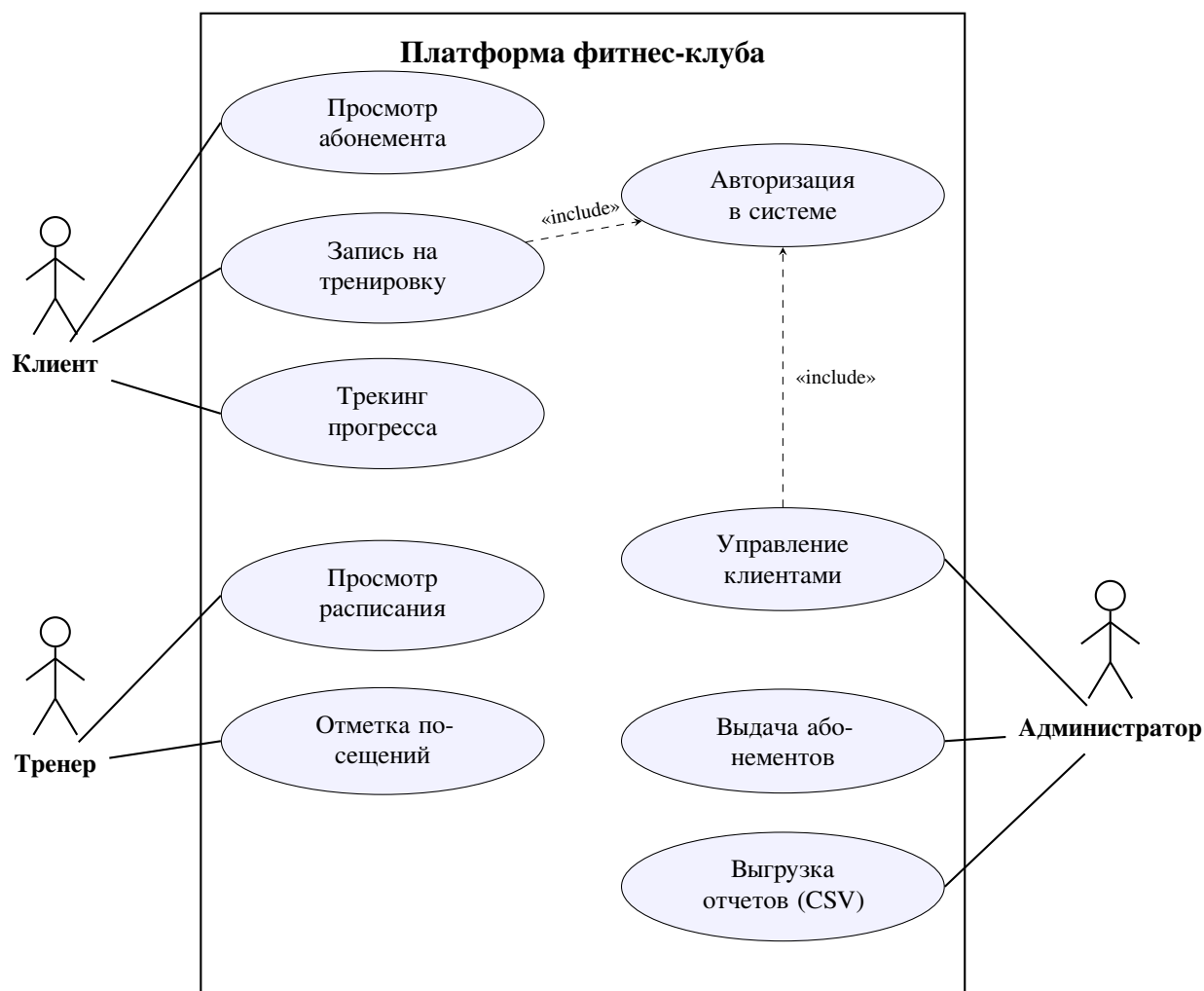


Рисунок 2.1 – Диаграмма вариантов использования (Use Case)

В системе реализована иерархия акторов: базовая абстракция берет на себя общие прецеденты авторизации и регистрации, а остальные роли наследуют эти права. Бизнес-логика изобилует сложными зависимостями. Разработанная поведенческая модель послужит надежным фундаментом для следующего этапа — проектирования физической структуры реляционной базы данных.

2.3 Проектирование модели данных

При проектировании инфологической модели были выделены следующие ключевые сущности:

- **users** — таблица для реализации системы авторизации и управления ролями.
- **workouts** — таблица для формирования сетки расписания и хранения информации о доступных тренировках.
- **memberships** и **bookings** — таблицы для реализации бизнес-логики контроля абонементов и записей на занятия.
- **progress_logs** — таблица для хранения физиологических параметров клиентов.

Особое внимание при проектировании было уделено архитектурной проблеме записей: один абонемент может покрывать множество различных занятий, поэтому статус посещения был вынесен на уровень позиции записи (**bookings**).

Логическая структура базы данных со связями типа «один-ко-многим» представлена на ER-диаграмме (рисунок 2.2).

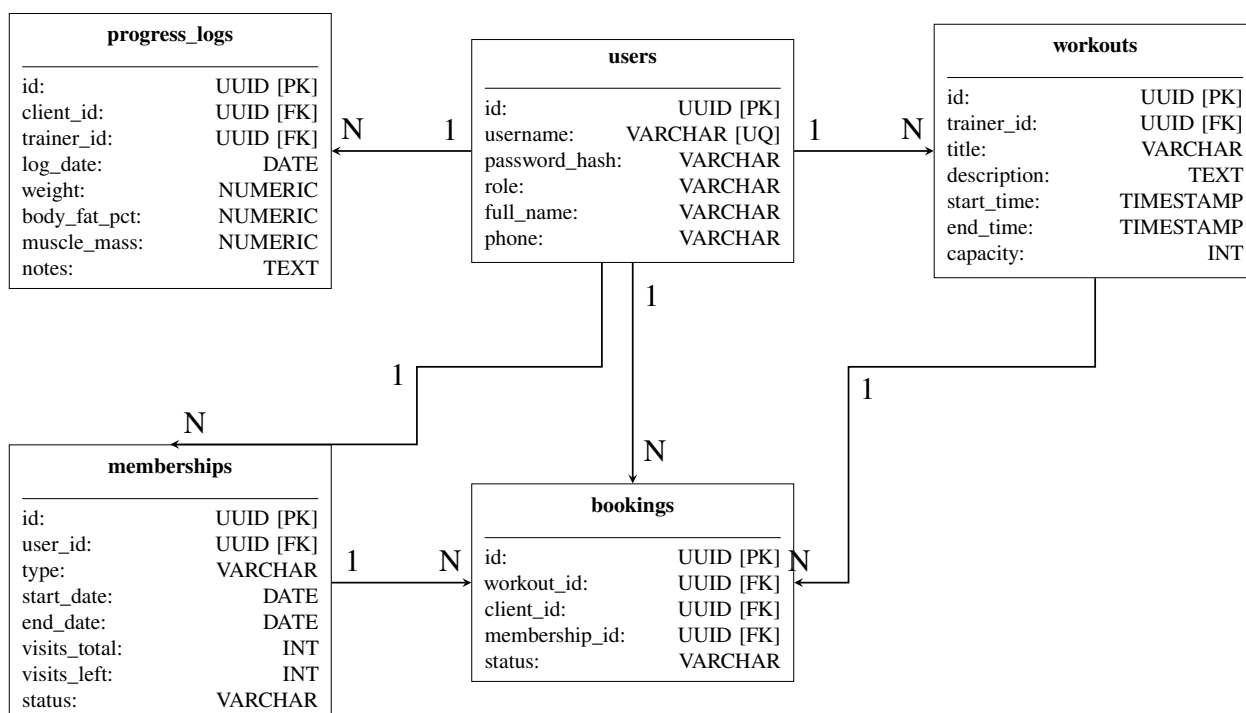


Рисунок 2.2 – ER-диаграмма базы данных

Спроектированная реляционная модель исключает дублирование данных и полностью готова к интеграции с серверной частью приложения через ORM SQLAlchemy.

2.4 Разработка макетов страниц

Для визуализации пользовательского интерфейса и проектирования путей взаимодействия были разработаны низкодетализированные структурные макеты основных страниц (Wireframes). Данный подход позволяет сфокусироваться на эргономике, расположении элементов управления и информационной архитектуре до начала этапа программной верстки.

Главная страница является точкой входа для клиентов. В верхней части расположена глобальная панель навигации с модулем поиска, кнопками авторизации и личного кабинета. Ниже расположен рекламный блок с призывом к действию. Ключевым элементом интерфейса является панель направлений, позволяющая клиенту отсеивать занятия по заданным критериям. Основное пространство занимает модульная сетка популярных тренировочных программ. Макет главной страницы представлен на рисунке 2.3.



Рисунок 2.3 – Макет главной страницы

Карточка тренировки разработана с акцентом на детальное представление услуги. Макет (рисунок 2.4) включает крупную область для просмотра фотографий зала или процесса занятий. В информационной панели, помимо цены и названия, предусмотрен вывод имени тренера и динамических селекторов (выбор уровня и времени). Ниже располагаются блоки с текстовым описанием, характеристиками программы и рекомендательной системой похожих занятий из той же категории.



Рисунок 2.4 – Макет карточки тренировки

Личный кабинет клиента (рисунок 2.5) спроектирован для удобного управления своим профилем, абонементом и записями на тренировки. Левая боковая панель обеспечивает быструю навигацию по разделам. Основная рабочая область содержит блоки с контактной информацией, статусом текущего абонента, таблицей ближайших записей и функциональной заглушкой для визуализации графика спортивного прогресса пользователя.

Таким образом, завершен этап архитектурного планирования и подготовлена исчерпывающая база для программной реализации серверной и клиентской частей системы в следующей главе.

CLUB LOGO

Профиль

Выйти

Меню

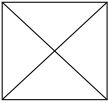
Профиль

Мой абонемент

Мои записи

Мой прогресс

Личный кабинет



Иванов Иван Иванович

Телефон: +7 (999) 000-00-00

Email: ivanov@mail.com

Мой абонемент

Классический (12 занятий)

Осталось занятий: 8

Продлить

Ближайшие записи

Дата и Время	Тренировка	Статус
25.06.2026, 18:00	Кроссфит	Подтверждено

Отменить

График прогресса

График динамики веса

Соцсети

Работаем с 2026 года

Контактные данные

Рисунок 2.5 – Структурный макет личного кабинета клиента

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

3.1 Разработка базовой структуры клиентской части приложения

Инициализация и сборка клиентской части веб-приложения была выполнена с помощью современного инструмента Vite в связке с библиотекой **React.js**, что полностью соответствует требованиям, определенным на этапе анализа предметной области. Данный выбор обеспечил высокую скорость перезагрузки модулей при разработке (HMR) и глубокую оптимизацию итогового бандла. Для стилизации интерфейсов применен фреймворк **Tailwind CSS**, реализующий концепцию Utility-First. Это позволило отказаться от громоздких каскадных таблиц стилей в пользу атомарных классов и в точности реализовать спроектированные в Главе 2 высокотехнологичные макеты (дашборды с кольцами активности и графиками).

Архитектура клиентской части построена на основе строгого компонентного подхода. Исходный код был логически разделен на следующие директории:

- `components` — переиспользуемые изолированные элементы пользовательского интерфейса (навигационные панели, карточки тренировок, виджеты прогресса, модальные окна выгрузки отчетов);
- `pages` — уникальные страницы-компоненты (личный кабинет клиента, панель управления администратора, дашборд расписания для тренеров);
- `api` — модуль инкапсуляции сетевых запросов и настройки связи с серверной частью.

Маршрутизация в одностраничном приложении (SPA) реализована с помощью библиотеки `react-router-dom`. Был спроектирован единый диспетчер маршрутов, обеспечивающий плавные переходы по разделам личного кабинета без перезагрузки браузера.

Ключевым элементом базовой структуры является централизованный HTTP-клиент на базе библиотеки **Axios**. Для обеспечения безопасности и прозрачной работы с сессиями был реализован механизм перехватчиков запросов (`interceptors`). Данный архитектурный паттерн автоматически извлекает токен доступа из локального хранилища браузера (`localStorage`) и внедряет

его в заголовки авторизации перед каждой отправкой запроса на сервер (см. листинг 1), исключая дублирование кода.

Листинг 1 – Реализация перехватчика Axios с автоопределением среды окружения

```
1 import axios from 'axios';
2
3 // Динамическое определение адреса для локальной разработки и деплоя
4 const isLocalhost = window.location.hostname === 'localhost'
5   ? 'http://localhost:5000/api'
6   : 'https://pulsecore-vnk4.onrender.com/api';
7
8
9 const api = axios.create({ baseURL });
10
11 // Перехватчик: автоматически вставляет токен авторизации
12 api.interceptors.request.use((config) => {
13   const token = localStorage.getItem('token');
14   if (token) {
15     config.headers.Authorization = `Bearer ${token}`;
16   }
17   return config;
18 });
19
20 export default api;
```

3.2 Разработка серверной части, реализация аутентификации и авторизации

В соответствии с заложенной архитектурой, серверная часть веб-приложения была разработана на платформе **Node.js** с использованием веб-фреймворка **Express**. Взаимодействие с реляционной системой управления базами данных PostgreSQL реализовано с помощью ORM-библиотеки **Sequelize**. Это позволило абстрагироваться от прямых SQL-запросов, использовать объектно-ориентированный подход к таблицам (users, memberships, workouts) и гарантировать защиту от SQL-инъекций.

Для организации надежной системы авторизации применен протокол **JSON Web Tokens (JWT)**. Данный подход обеспечивает Stateless-архитектуру: сервер не хранит активные сессии в оперативной памяти, что кратно повышает

масштабируемость и отказоустойчивость платформы. Пароли пользователей не сохраняются в БД в открытом виде — перед записью они проходят криптографическое хеширование алгоритмом bcrypt.

При успешной проверке учетных данных алгоритм генерирует подписанный JWT-токен, в полезную нагрузку (payload) которого внедряются идентификатор пользователя и его роль (role).

Специфика фитнес-платформы требует строгого разделения прав доступа между клиентами, тренерами и администраторами. Для решения этой задачи в Express реализованы специализированные функции промежуточной обработки (Middlewares). Модуль `roleCheckMiddleware` перехватывает входящий запрос, декодирует токен и проверяет наличие необходимых прав. Если клиент пытается обратиться к эндпоинту администратора (например, выгрузке отчетов), система автоматически прервет обработку и вернет HTTP-статус 403 Forbidden, обеспечивая надежный контур безопасности.

3.3 Реализация CRUD-интерфейсов для пользователей, абонементов и тренировок

В приложении был реализован комплекс модулей, обеспечивающих CRUD-операции (создание, чтение, обновление, удаление) для всех ключевых сущностей спроектированной базы данных. Логика управления сегментирована в зависимости от ролевой модели.

Для администратора спроектированы эндпоинты управления таблицами `users` и `memberships`. Реализован интерфейс модерации, представляющий собой агрегированную таблицу с данными о клиентах и статусах их абонементов. Администратор наделен правами редактирования профилей, оформления новых абонементов и обнуления визитов. Благодаря настроенному каскадному удалению на уровне PostgreSQL, при удалении учетной записи автоматически очищаются связанные с ней записи в таблицах `bookings` и `progress_logs`.

Для тренера реализован интерфейс управления расписанием (`workouts`). При создании тренировочного слота тренер задает название, время начала и лимит участников. Тренер также имеет доступ к чтению записей из таблицы `progress_logs` для своих клиентов.

Модуль файлов: Создание карточки прогресса подразумевает загрузку медиафайлов. Этот функционал реализован через отдельный эндпоинт с

использованием библиотеки `multer` для обработки `multipart/form-data`. Файл сохраняется в защищенное хранилище с валидацией расширений, а в БД (`photo_url`) записывается строковая ссылка с поддержкой физического удаления файла при удалении замера.

3.4 Реализация системы записи на занятия и бизнес-логики списания визитов

Ключевым техническим вызовом стала реализация модуля управления записями (`bookings`) и списания остатка занятий. Бизнес-логика оформления записи инициируется клиентом через графический интерфейс. Обработка запроса на стороне сервера представляет собой строгую ACID-транзакцию, выполняющуюся в несколько этапов:

1. **Проверка консистентности:** сервер проверяет вместимость группы.
2. **Валидация абонемента:** система проверяет, что срок действия не истек, а остаток занятий (`visits_left`) больше нуля.
3. **Списание остатка:** в рамках транзакции система декрементирует значение `visits_left` в таблице `memberships` на 1 (одно занятие).
4. **Фиксация записи:** в таблицу `bookings` добавляется новая строка со ссылками на клиента и тренировку. Транзакция фиксируется (`COMMIT`).

При успешном завершении транзакции графический интерфейс клиента (кольца активности в дашборде) динамически пересчитывает оставшийся лимит, а занятие появляется в расписании тренера. При отмене записи со стороны клиента срабатывает обратная логика: статус меняется на отмененный, а занятие транзакционно возвращается на счет абонемента.

3.5 Разработка модуля отчетности и работы с файлами

Для автоматизации работы руководства был спроектирован модуль коммерческой аналитики, интегрированный в десктопную панель администратора. Задачей модуля является предоставление агрегированных финансовых показателей и генерация отчетной документации (выгрузки).

Расчет ключевых показателей (KPI), таких как общее число активных клиентов, рост выручки и посещаемость, выполняется исключительно на

стороне базы данных с применением агрегатных SQL-функций (например, `SUM()`, `COUNT(DISTINCT)`). Это переносит вычислительную нагрузку с Node.js на ядро PostgreSQL, обеспечивая мгновенную отрисовку метрик в верхней панели интерфейса администратора.

Бизнес-логика виджета экспорта отчетов поддерживает формирование файлов в формате CSV. Логика генерации реализована с учетом оптимизации ввода-вывода и экономии дискового пространства сервера:

1. **Сбор данных:** сервер выполняет SQL-запрос с объединениями (JOIN) таблиц `users`, `memberships` и `bookings`.
2. **Виртуализация буфера:** вместо физического сохранения сформированного документа на жестком диске сервера, используются потоки данных в оперативной памяти.
3. **Кодирование:** для корректного отображения кириллицы в табличных редакторах (Excel) в начало потока принудительно записывается маркер последовательности байтов (BOM).
4. **Стриминг:** готовый буфер отправляется клиенту через HTTP-ответ в виде скачиваемого файла (устанавливаются заголовки `Content-Disposition: attachment`).

Принятые архитектурные решения обеспечили высокую производительность приложения, гарантируя клиентам быстрый отклик интерфейса, а руководству клуба — безопасный и мгновенный экспорт аналитических данных.

ЗАКЛЮЧЕНИЕ

В результате проведенного исследования и практической разработки подтверждена гипотеза о том, что внедрение специализированного веб-приложения с изолированными личными кабинетами позволяет существенно повысить эффективность работы фитнес-клубов и лояльность клиентов.

На основе полученных результатов можно сформулировать следующие **выводы**:

1. Использование современного стека технологий (React.js, Node.js, Tailwind CSS) обеспечивает необходимую архитектурную гибкость и позволяет реализовать отзывчивый пользовательский интерфейс в виде одностраничного приложения (SPA) без лишних перезагрузок страниц.
2. Проектирование реляционной модели данных в PostgreSQL и использование ORM-библиотеки Sequelize гарантирует целостность системы. Внедрение строгих транзакций при обработке записи на тренировки полностью исключает риск несанкционированного перерасхода занятий по абонементу (защита от состояния гонки).
3. Разработанная трехуровневая ролевая модель (Клиент, Тренер, Администратор) на базе протокола JWT-авторизации обеспечивает надежное разграничение прав доступа и защищает коммерческую информацию клуба от несанкционированного доступа.
4. Реализованный модуль аналитики, переносящий вычислительную нагрузку на агрегатные функции СУБД, демонстрирует высокую производительность при формировании финансовых отчетов, а использование виртуальных буферов оперативной памяти делает экспорт в форматах Excel, CSV и PDF быстрым и безопасным для сервера.

Рекомендации и предложения. Для практического применения разработанного решения PulseCore в деятельности малых фитнес-клубов предлагается использовать модель облачного развертывания (SaaS). Это обеспечит бесперебойный доступ к интерфейсу управления как со смартфонов клиентов, так и с рабочих станций администраторов без дополнительных затрат на ИТ-инфраструктуру.

Перспективы углубления темы. В дальнейшем исследовании планируется развитие функциональности системы за счет внедрения модуля онлайн-эквайринга для прямой оплаты абонементов банковскими картами, а также интеграции push-уведомлений для напоминания клиентам о предстоящих тренировках и скором истечении срока действия тарифа.

При разработке пояснительной записки к курсовому проекту строго соблюдались требования [1], определяющие структуру и правила оформления научно-исследовательских работ.

Материалы проекта:

- рабочий экземпляр приложения (хостинг): <https://pulsecore-frontend.onrender.com/>;
- репозиторий с исходным кодом (GitHub): <https://github.com/konnan2282/pulsecore>.

СПИСОК ИСТОЧНИКОВ ЛИТЕРАТУРЫ

- [1] ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления [Текст]. — 2017.
- [2] PostgreSQL Documentation: JSON Types [Electronic resource]. — [S. l. : s. n.]. — URL: <https://www.postgresql.org/docs/current/datatype-json.html> (online; accessed: 29.04.2026).
- [3] React Documentation: Quick Start [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://react.dev> (дата обращения: 29.04.2026).
- [4] FastAPI Documentation: Professional API development [Electronic resource]. — [S. l. : s. n.]. — URL: <https://fastapi.tiangolo.com> (online; accessed: 29.04.2026).
- [5] Tailwind CSS Documentation: Utility-First Fundamentals [Electronic resource]. — [S. l. : s. n.]. — URL: <https://tailwindcss.com/docs> (online; accessed: 29.04.2026).
- [6] Проектирование B2B-интерфейсов: паттерны, дашборды и аналитика [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://uxpub.ru/b2b-dashboard-design> (дата обращения: 04.05.2026).
- [7] Репозиторий GitHub с кодом проекта [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://github.com/konnan2282/pulsecore> (дата обращения: 04.06.2026).
- [8] Сайт, опубликованный на хостинге [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://pulsecore-frontend.onrender.com/> (дата обращения: 13.06.2026).